F24-186-D-Reagent

Project Team

Muhammad Omais Afzal 21I-1369
Ahsan Wasim 21I-0440
Muhammad Hamdan Ali 21I-0754

Session 2021-2025

Supervised by

Dr. Javaria Imtiaz

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2024

Contents

List of Figures

List of Tables

CHAPTER 1

INTRODUCTION

Reagent is an assistant that utilizes AI to detect bad code and security vulnerabilities in Python. While developing software, security, maintainability, and clean and efficient code are crucial for the reliability of the applications. Identifying these problems can be a hurdle and time-consuming. Conventional tools that do static analysis often fail to identify problems that can cause issues in the future.

The adoption of Python is widespread in the software industry and academia, and it has huge code bases and usage. Developers require more robust tools than simple syntax checking or error detection. Tools that can support structural issues and reduce false positives security issues are necessary. Moreover, students have a hard time learning to code correctly by making mistakes like hard coding, security flaws, re-usability, and coding according to industry standards. Reagent fills this gap by using LLMs to cater to this problem, as it can easily understand the intent and the context of the code.

Reagent understands code context and semantics, allowing it to detect a broader range of issues, including complex code smells and subtle vulnerabilities that might go unnoticed. Furthermore, Reagent integrates seamlessly into development environments like PyCharm provides real-time feedback and actionable recommendations, making the process of maintaining high code quality effortless and efficient.

By addressing these gaps, Reagent empowers developers and students to write better code with less effort, ultimately leading to more secure, maintainable, and high-performing Python applications.

1.1 Problem Statement

Quality according to the industry standards and staying up to par while maintaining high-quality applications that work seamlessly with little to no downtime is not achievable

due to a lack of developer knowledge and students not adjusting to industry standards of coding. This leads to increased costs of maintaining the code and leaves it vulnerable to attackers that result in data theft or completely choking the system. Without effective tools that ensure quality, developers cannot maintain quality; even if there are good developers, there is always room for error. Static code analysis tools exist, but they lack understanding of the code's intent and context. Current static analysis tools provide some level of assistance, but they are often limited in scope, lack context awareness, and frequently produce a high number of false positives or overlook critical issues altogether. Continuous updates with the latest data need to be integrated into a tool that can understand the context.

1.2 Scope

Reagent aims to provide a coding interface in the PyCharm IDE for Python code that performs live code standard detection along with vulnerability and threat detection for novice programmers. It aims to help programmers learn the industry standard of coding and to rid their code of any vulnerabilities. Reagent will include a simple and effective AI model that will suggest code-style fixes and vulnerabilities. For example, if the AI detects SQL-based vulnerability will highlight the line in the code and then suggest a possible fix. Reagent will help detect coding structure mistakes according to the PEP8 standards.

Reagent will be powered by Llama 3 open-source AI model catered to specific coding styles and vulnerabilities. Being a plugin, it will provide seamless integration into PyCharm is a popular IDE for Python programmers. Along with code style and threat detection, Reagent will also provide a code narration feature that will create a short summary of the code. The narration will help other programmers understand the motive behind the code and ease the process of re-usability. Writing clean code with minimal vulnerabilities is one thing, but if code efficiency is compromised, the code becomes unusable. Therefore, Reagent will also help reduce the time complexity of the code so that it remains efficient and fast.

Reagent will provide live suggestions for code fixes by using popups and in-line suggestions, much like the GitHub co-pilot UI. Additionally, the plugin will provide a user-friendly interface for a seamless coding experience. By narrowing the focus to code standards and vulnerabilities, Reagent aims to address the specific challenges beginners face while coding, making it an accessible and specialized tool for users across different industries.

1.3 Modules

Following are the main modules for Reagent:

1.3.1 Training Python Standards and Security Threats Model

This module aims to develop a model using PyTorch based on LLaMA 3. The model will be designed to test Python code against PEP8 standards and check for common security threats. This approach is intended to improve code quality and security by identifying adherence to coding standards and detecting potential vulnerabilities.

1. **Model:** A fine-tuned version of the LLaMA 3 model will be trained to assess Python code quality and detect security threats.
2. **Data Sources:** Data will be sourced from Hugging Face and Python codebases.
3. **Evaluation Criteria:** The model will be evaluated based on its ability to identify Python coding standards and detect security threats. It will be integrated into PyCharm for real-time analysis.

1.3.2 Designing and Implementing a PyCharm Plugin Interface

This module will focus on developing the Reagent plugin frontend using Kotlin/Java, which will integrate seamlessly with PyCharm. The plugin will provide immediate feedback on code quality and security through an intuitive and interactive UI.

1. **Real-time Feedback:** The plugin will provide real-time feedback on coding standards and security vulnerabilities while code is being written.
2. **Full Integration:** It will feature seamless integration into the PyCharm UI, including suggestions, warnings, and detailed reports.
3. **Visualization and UX:** The plugin will offer inline annotations and a user-friendly interface for generating code reports.

1.3.3 Testing and Analysis

This module will address the performance analysis and user feedback for the Reagent plugin to ensure it meets expectations.

1. **Unit Testing:** The accuracy of model predictions and plugin functionality will be tested.
2. **Integration and Performance Testing:** The plugin's integration with PyCharm will be assessed for smooth operation, and performance metrics will be measured.
3. **User Testing:** Feedback from beta users will be collected to evaluate usability, prediction accuracy, and overall user experience.

1.4 User Classes and Characteristics

The following table contains the classes of users for the Reagent project:

User Class	Description
Developer	Developers are the primary users of Reagent, using the plugin to evaluate Python code for PEP8 violations and security vulnerabilities. They receive real-time feedback while writing code and summaries of detected issues when finished. The tool assists developers in maintaining clean, secure, and efficient code.
Student	Students learning Python can use Reagent to get instant suggestions on writing better, more maintainable code. It helps them understand code standards like PEP8 and recognize potential security threats as they learn to code.
Security Analyst	Security analysts use Reagent to ensure Python applications meet security standards. They focus on detecting and mitigating security vulnerabilities in Python code, ensuring that the applications are secure before deployment.
Project Manager	Project managers use Reagent to assess the quality of the code written by their team members. They review reports generated by the plugin to ensure that the team's code adheres to PEP8 standards and does not have security vulnerabilities.

CHAPTER 2

PROJECT REQUIREMENTS

This chapter describes the functional and non-functional requirements of the project.

2.1 Event Response Table

Event ID	Event	Response
1	Developer writes code in PyCharm	Code is monitored by the Reagent plugin for analysis.
2	Plugin checks for PEP8 violations	Detected PEP8 violations are displayed with location and details.
3	Plugin checks for vulnerabilities	Detected security vulnerabilities are displayed with details on potential threats and impact.
4	Developer receives live code suggestions	Live code improvement suggestions are displayed as the developer types.
5	Developer presses the "Summary" button after finishing the code	A summary report is generated, outlining all PEP8 violations, security vulnerabilities, and solutions applied during the session.
6	Developer reviews the summary report	Developer evaluates the report and applies final changes if needed.

Table 2.1: Event-Response Table for Reagent Plugin

2.2 Functional Requirements

Reagent aims to provide free PEP8 code standard and vulnerability detection for programmers, while also providing code summary generation. Here are a few functional

requirements that will enable us to achieve this goal:

2.2.1 Module 1: Training Python Standards and Security Threats Model

1. Data Ingestion and Pre-processing:
    - Utilize data from hugging face and open-source Python code-bases for model fine-tuning.
    - Pre-process the data to extract code samples best fit for PEP8 standard detection and to check for vulnerabilities.
2. Model Training:
    - Use LLaMA 3 large language model to check Python code for any PEP8 code standard violations and for vulnerabilities.
3. Model Evaluation Metrics:
    - Define and use evaluation metrics for the model, such as precision.
    - Improve and enhance the model based on the metric scores

2.2.2 Module 2: Designing and Implementing a PyCharm Plugin Interface

1. Real-Time Code Analysis:
    - Integrate the plugin with PyCharm to provide real-time suggestions for code fixes and vulnerability detection.
    - Display the suggestions as alerts or popup messages where an error is found.
2. Seamless integration with PyCharm:
    - Ensure seamless integration with PyCharm for live code detection and code fixes.
    - Add a button toggle for code summary generation.
    - Implementation of visual indicators like line highlighting, popups, alerts, and inline warnings.

2.3 Non-Functional Requirements

This section defines the quality attributes of the Reagent system, specifying quantitative and verifiable conditions for the software's reliability, usability, performance, and security. These requirements ensure Reagent meets its intended functionality while maintaining a high-quality standard.

The numbers that are mentioned below are the aimed numbers. Actual performance might vary due to code, resource, or performance limitations.

2.3.1 Reliability

Reliability defines how often software fails and the expected behavior under various conditions. It is frequently measured using the Mean Time Between Failures (MTBF) and the consequences of software failure.

- The reagent shall have an MTBF of at least 500 hours, meaning that the plugin should operate without failure for at least 500 hours on average.
- In the event of a failure (e.g., failure to detect code standards or vulnerabilities), the system must log the error and attempt to self-correct or notify the user within 5 seconds.
- The system shall detect potential failures (such as plugin crashes or unexpected behavior in code analysis) and initiate an automatic recovery process to restart the service without losing user data.
- All failure events must be recorded in a log file, containing detailed information regarding the failure cause and context for future diagnostics.

Error Detection and Recovery

- If a failure occurs, the assistant will display a popup warning in the PyCharm interface, allowing the user to correct the error manually or retry the operation.
- Notifications will be sent to the users in case of errors.

2.3.2 Usability

Usability ensures the system is easy to use, promotes efficiency, and avoids errors, making it accessible to a broad range of users, including beginners.

- Reagent shall provide feedback to the user within 2 seconds of detecting a code error or vulnerability. This feedback will include recommendations for code corrections.
- The system shall offer tooltips and inline guidance to assist users in understanding detected errors, allowing them to fix code efficiently.
- The user interface shall be intuitive, allowing a novice user to learn and use the tool within 15 minutes without extensive training.
- The system shall be accessible to users with different needs, providing customizable font sizes, themes (light/dark), and keyboard shortcuts.
- Reagent shall ensure error recovery by allowing users to undo code suggestions or corrections with one-click actions.

2.3.3 Performance

Performance requirements outline the system's response time and resource consumption.

- Reagent shall process code analysis requests for PEP8 standard and vulnerability detection within 1 second for every 100 lines of code.
- Real-time suggestions and popups should appear with a delay of 1 second after detecting an issue in the user's code.
- The plugin shall consume no more than 15% of CPU resources during regular operation and less than 500MB of memory in idle mode to ensure smooth integration with PyCharm.
- The plugin must maintain responsiveness even in large codebases (up to 10,000 lines) and not introduce noticeable lag or slow down PyCharm's performance.

2.3.4 Security

Security requirements specify how the system protects itself and its data, ensuring confidentiality, integrity, and availability.

- The Reagent plugin shall ensure all data (e.g., code samples, suggestions, and processed) are kept locally within the user's machine and will not be transmitted externally without user consent.

- The plugin shall be protected from unauthorized access by implementing role-based access controls, ensuring only authorized personnel can modify plugin configurations.
- Reagent shall resist common attacks such as code injection or plugin tampering by validating all inputs and restricting access to sensitive operations.
- The plugin shall be designed to handle malicious inputs securely, ensuring that any invalid or corrupted data fed into the system does not result in failure or vulnerability exploitation.
- Security audit logs shall be maintained for a minimum of 30 days, documenting all key actions of the user, such as vulnerability detection or attempted attacks, for future analysis.

2.4 Domain Model

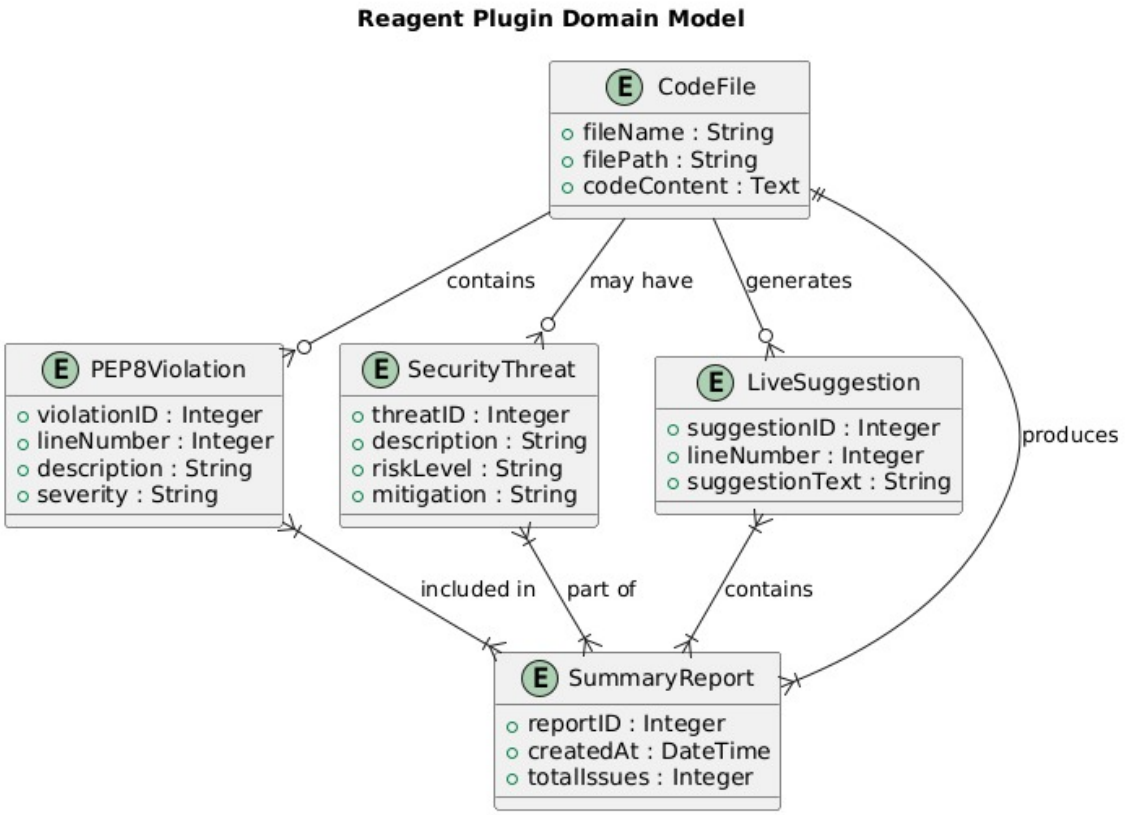

Figure 2.1: Reagent Domain Model

CHAPTER 3

SYSTEM OVERVIEW

This chapter provides a comprehensive overview of the Reagent plugin system, including its functionality, context, and design. The Reagent plugin is designed to assist Python developers by offering real-time detection of PEP8 violations, security vulnerabilities, and live code suggestions, all within the PyCharm IDE.

3.1 Architectural Design

The Reagent plugin is designed using a client-server architecture to support real-time detection of PEP8 violations, security vulnerabilities, and live code suggestions while maintaining efficient processing. This modular program structure allows for a clear separation of concerns between the client-side interface and the backend analysis system.

3.1.1 Modular Program Structure

1. Client-Side (PyCharm IDE with Reagent Plugin) The developer interacts with the system through the PyCharm IDE. As the developer writes Python code, the Reagent Plugin sends the code to the backend server for analysis in real-time. The Reagent Plugin receives analysis results, including PEP8 violations, security vulnerabilities, and live suggestions, and displays them to the developer immediately.The plugin also allows the developer to generate a summary report detailing all detected issues.

2. Server-Side (Backend Server) The backend server handles the core analysis logic and machine learning models responsible for detecting PEP8 violations and security vulnerabilities. It consists of:

- PEP8 Violation Detector: Detects formatting violations based on PEP8 standards.
- Security Threat Detector: Identifies potential security vulnerabilities in the code.
- Live Code Suggestion Engine: Provides real-time suggestions to improve the code.
- Summary Generator: Compiles a summary report of the analysis results.

3.1.2 Relationships Between Modules

The interaction between the client (Reagent Plugin) and the server is orchestrated as follows:

- The Developer writes code, which is processed by the Reagent Plugin.
- The plugin sends the code to the backend server for analysis.
- The backend runs checks for PEP8 violations and security threats and returns the results to the plugin.
- The plugin then displays these results and suggestions to the developer in real-time.

3.1.3 Box and Line Diagram

In the initial design stage, we use a Box and Line Diagram to represent the system's structure. Below is the diagram showing the major subsystems and their connections.

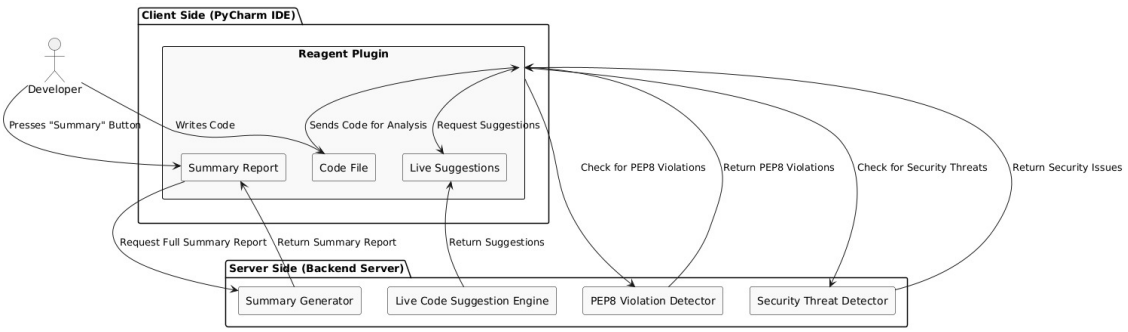

Figure 3.1: Box and Line Diagram of Reagent Plugin

3.1.4 Final Architecture

The finalized architecture follows a Client-Server Model, where the Reagent Plugin acts as the client, and the Backend Server performs all code analysis. This structure ensures efficient processing while allowing real-time interactions for the developer.

The final architecture is detailed as follows:

- Client Side: Developer interface (PyCharm IDE) and the Reagent Plugin.
- Server Side: PEP8 Violation Detector, Security Threat Detector, Live Code Suggestion Engine, and Summary Generator.

This architectural design enables the Reagent plugin to provide seamless feedback to developers while maintaining modularity for easy future enhancements.

3.2 Design Models

This section presents the design models applicable to the Reagent plugin project. Each model is critical in understanding the system's architecture and functionality. The following design models have been developed following a procedural approach:

3.2.1 Activity Diagram

The Activity Diagram illustrates the workflow of the Reagent plugin, showing how the developer interacts with the plugin in real-time. It captures the sequence of activities from writing code to receiving feedback.

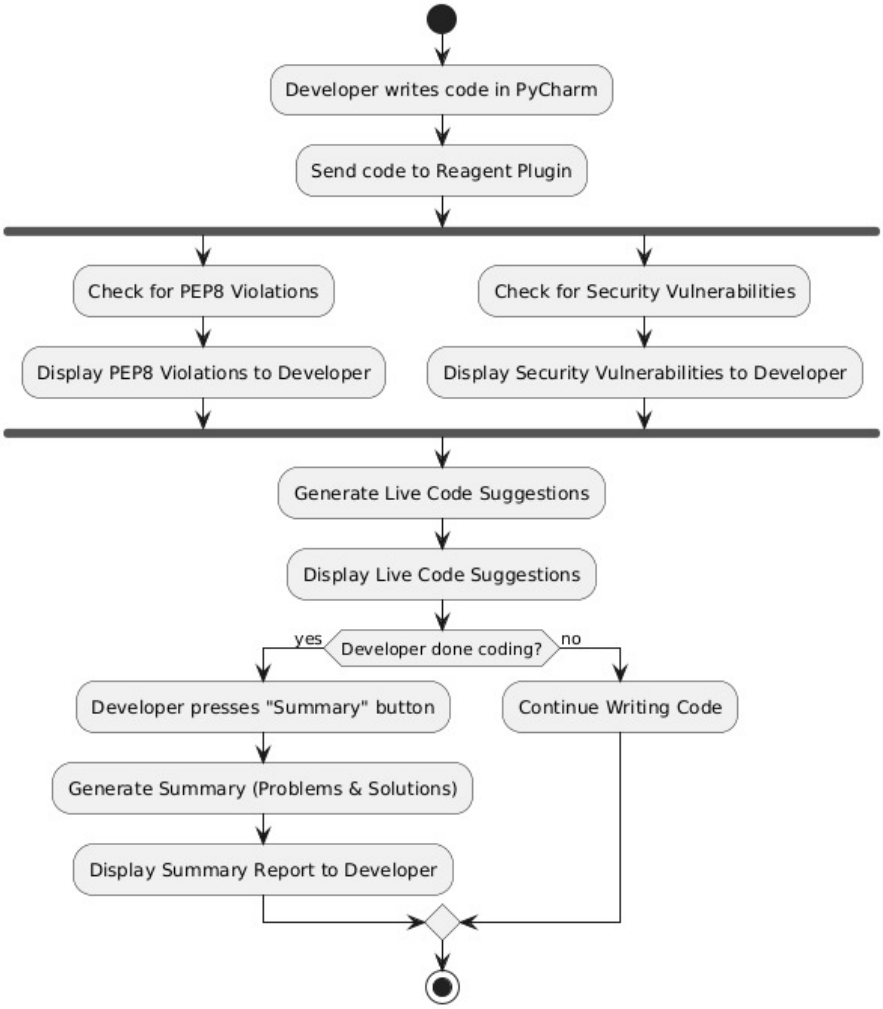

Figure 3.2: Activity Diagram for Reagent Plugin

3.2.2 Data Flow Diagram (DFD)

The Data Flow Diagram represents how data flows through the Reagent plugin system. It includes interactions between the developer, the PyCharm IDE, the Reagent Plugin, and the Backend Server. The DFD is extended to 2-3 levels to detail all processes, sources, sinks, and data stores.

- Level 1 DFD The high-level DFD shows the main components involved in the Reagent plugin.

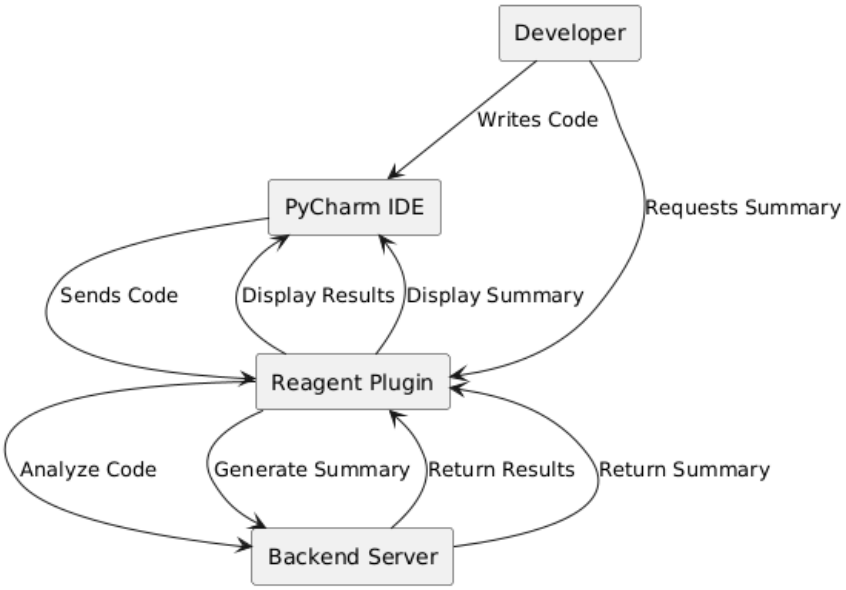

Figure 3.3: Level 1 Data Flow Diagram

- Level 2 DFD The detailed DFD focuses on backend processing, showing how data flows through the components responsible for analysis.

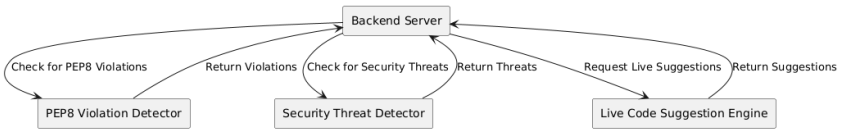

Figure 3.4: Level 2 Data Flow Diagram

3.2.3 System-Level Sequence Diagram

The System-Level Sequence Diagram illustrates the order of operations between the developer, the Reagent Plugin, and the Backend Server during code analysis. It provides a clear view of the interactions and data flow during development.

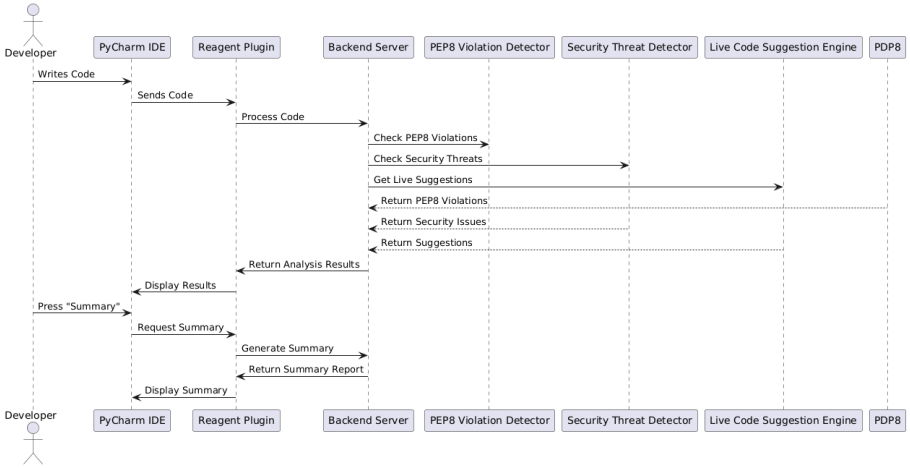

Figure 3.5: System-Level Sequence Diagram

3.2.4 State Transition Diagram

The State Transition Diagram depicts the various states of the Reagent plugin and how the system transitions between these states based on user interactions and system events. It highlights the handling of events such as writing code, checking for violations, and generating summaries.

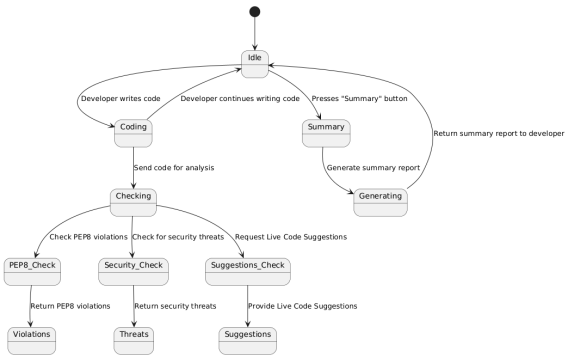

Figure 3.6: State Transition Diagram

3.2.5 Entity-Relationship Diagram (ERD)

The Entity-Relationship Diagram (ERD) provides a visual representation of the key entities within the Reagent plugin system and their relationships. This diagram helps in understanding how different components interact with each other in the context of data storage and processing.

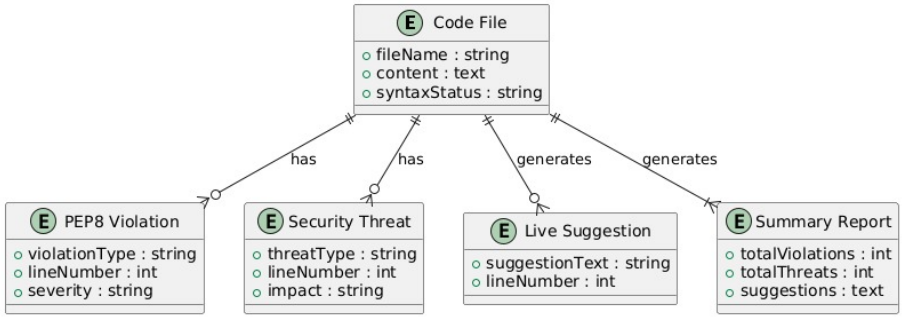

Figure 3.7: Entity-Relationship Diagram for Reagent Plugin

3.2.5.1 Description of Entities and Relationships

The main entities in the ERD are:

- **Code File**: Represents the Python code being analyzed.
- **PEP8 Violation**: Represents violations detected based on PEP8 standards.
- **Security Threat**: Represents potential security vulnerabilities found in the code.
- **Live Suggestion**: Contains real-time suggestions for improving the code.
- **Summary Report**: Compiles a summary of all analysis results.

The relationships among these entities indicate that a Code File can have multiple PEP8 Violations and Security Threats, and can generate multiple Live Suggestions and a Summary Report. This structure is crucial for maintaining an organized approach to code analysis and improving developer productivity.

3.3 Data Design

3.3.1 Abstract Syntax Tree (AST)

- **Purpose**: Used for parsing Python code and analyzing its structure.
- **Description**: The AST is a hierarchical tree structure that represents the syntactical structure of Python code. It's useful for performing static analysis, detecting code smells, and finding PEP8 violations. AST nodes can represent code elements such as functions, loops, variables, etc.

  Example: Analyzing function definitions, loops, and variable assignments to detect PEP8 violations or vulnerabilities like hard-coded credentials.

- **Structure**: Tree with nodes representing syntactic constructs in the source code.

3.3.2 Hash Maps (Dictionaries)

- **Purpose**: Used to store and retrieve key-value pairs efficiently.
- **Description**: Used to store the mapping between line numbers of code and corresponding issues (e.g., PEP8 violations or vulnerabilities). It can store settings for users, such as preferences or plugin configurations. Example: { line_number: ["PEP8 violation: Missing docstring", "Security flaw: SQL injection"] }
- **Structure**: Dictionary or HashMap for fast lookups, insertion, and deletion.

- **Purpose**: Store sequential data such as lines of code, user inputs, or logs.
- **Description**: Lists are used to hold sequential data, such as the lines of code being analyzed. Example: A list of all lines in the source code file being analyzed. Each entry might be indexed by the line number.
- **Structure**: Simple arrays or dynamic lists (in Python, list).

3.3.3 Tuple (Immutable Data)

- **Purpose**: Used to store immutable data such as analysis results that do not need to be modified.
- **Description**: Tuples can be used to store static or immutable data such as the initial analysis of code lines (PEP8 violations) or security threat metadata. Example: (line_number, violation_type, severity)
- **Structure**: A collection of immutable, ordered elements.

3.3.4 Linked List

- **Purpose**: Used for log tracking and version control.
- **Description**: A linked list can help in tracking changes to the code and the analysis process over time. It is useful for version control features, like undo/redo functionality, where each change is stored as a node in a linked list.
- **Structure**: The single or doubly linked list to track changes.

3.3.5 Bloom Filter

- **Purpose**: Efficiently check for common patterns or known vulnerabilities.
- **Description**: A **Bloom Filter** can be used to quickly determine if a certain pattern or vulnerability signature has already been detected without explicitly storing all the data. Example: Checking for common vulnerability signatures (e.g., SQL injection patterns) across large codebases.
- **Structure**: Probabilistic data structure for efficient membership testing.

3.3.6 Event Queue

- **Purpose**: Queue for real-time events such as feedback and suggestions.
- **Description**: Used to manage real-time events like user actions or plugin-triggered alerts, ensuring smooth interaction without overwhelming the user. **Example:** An event queue to handle real-time suggestions, warnings, and logs from the plugin as the user writes code.
- **Structure:** FIFO queue for sequential handling of events.

3.3.7 Data Flow

1. **Code Ingestion**
    - Code from PyCharm is input into the plugin.
    - AST is generated for structural analysis.
2. **Analysis and Storage** Issues such as PEP8 violations and vulnerabilities are stored in HashMaps for fast lookup. The severity of issues is stored in a Priority Queue to display the most critical errors first.
3. **Code Summaries** Code summaries and auto-complete suggestions can be stored and fetched from a Prefix Tree.
4. **Log and Error Tracking** Errors are stored in a Linked List for easy tracking and recovery.
5. **Training Data Storage** Training data is handled through Dictionaries and mapping code snippets to analyze results during the machine learning phase.

CHAPTER 4

CONCLUSION AND FUTURE WORK

4.1 Conclusion

Reagent is a powerful tool that can help novice programmers become professionals by helping them learn and adapt to write clean, efficient, and secure code. Reagent will use advanced LLMs (Large Language Models) to detect code standard violations and vulnerabilities. The plugin will then suggest code fixes via popup messages and alerts to ensure the code is fixed on the go. Reagent aims to help Python programmers; therefore, it will provide seamless integration with PyCharm. Reagent's functionality is underpinned by a well-thought-out data design that utilizes structures like Abstract Syntax Trees (ASTs) for code parsing and hash Maps for quick retrieval of analysis results for issue prioritization and summary generation. These choices ensure the plugin can efficiently handle real-time operations without compromising performance, even in large codebases. Additionally, the system's ability to track changes via Linked Lists and handle real-time events through an Event Queue ensures smooth user interaction and reliability. The non-functional requirements, such as reliability, usability, performance, and security, are addressed through specific, measurable goals, including low failure rates, intuitive user interfaces, fast response times, and robust security measures. Reagent will be capable of processing complex Python codebases while maintaining a low resource footprint, ensuring a seamless developer experience. Ultimately, Reagent empowers both novice and professional developers to write high-quality, secure, and efficient Python code. Its real-time feedback, coupled with deep learning-powered analysis, will significantly enhance productivity and code quality, bridging the gap between theoretical best practices and their application in everyday coding tasks.

Bibliography